

Requirements & Workload Specification

Report

Delivery 1

Moreno Barragán Jhonatan David

Gaitán Castañeda Luna Valeria

Jonathan Felipe Ochoa Silva

Ortiz Luna Nicolás Francisco

September 25, 2026

Contents

1	Executive Summary	3
1.1	Application Overview	3
1.2	Key Distributed Systems Challenges	3
2	System Description	5
2.1	Business Problem	5
2.2	Actors	7
2.3	Main Use Cases	9
2.4	System Boundaries	14
3	Functional Requirements	16
3.1	Account and Driver Management	16
3.1.1	FR-01 – Driver Onboarding and Verification	17
3.1.2	FR-02 – Driver Availability Management	18
3.2	Ride Request and Matching	18
3.2.1	FR-03 – Ride Request Creation	19
3.2.2	FR-04 – Driver Matching and Assignment	20
3.2.3	FR-05 – Fare and ETA Estimation	21
3.3	Trip Management	21
3.3.1	FR-06 – Trip Lifecycle Management	22
3.3.2	FR-07 – Real-Time Trip Tracking	23

3.4	Payment and Settlement	24
3.4.1	FR-08 – Fare Calculation and Payment Processing	24
3.5	Feedback and Safety	25
3.5.1	FR-09 – Rating and Feedback Management	25
3.5.2	FR-10 – Safety and Support Case Management	26
3.6	Functional Requirements Summary	27
4	Non-Functional Requirements	27
4.1	Performance and Capacity	28
4.1.1	NFR-01 – Expected Users	28
4.1.2	NFR-02 – Average Request Rate	28
4.1.3	NFR-03 – Peak Request Rate	29
4.1.4	NFR-04 – Peak Load Duration	29
4.1.5	NFR-05 – Response Time	30
4.2	Availability and Reliability	31
4.2.1	NFR-06 – System Availability	31
4.3	Data Management	31
4.3.1	NFR-07 – Initial Data Volume	31
4.3.2	NFR-08 – Annual Data Growth	32
4.4	Geographic Scope	32
4.4.1	NFR-09 – Global Geographic Coverage	32
4.5	Non-Functional Requirements Summary	33
5	Workload Model	34
5.1	Workload Parameters	35
5.2	Workload Assumptions	35
5.3	Workload Scenarios	36

1 Executive Summary

1.1 Application Overview

This project defines the requirements and workload model for building a global ride-sharing platform from scratch, inspired by the core capabilities of large-scale services such as Uber. The platform is designed as a two-sided marketplace that connects riders requesting transportation with independent drivers providing vehicle capacity in real time. Rather than owning or directly operating the vehicles, the system is responsible for coordinating the interaction between both sides of the marketplace, from ride request and driver matching to trip tracking, payment settlement, feedback, and safety support.

The platform must provide a complete end-to-end ride experience. A rider should be able to authenticate, specify a pickup location and destination, receive an estimated fare and arrival time, request a ride, and be matched with an eligible nearby driver. The driver must be able to complete the onboarding and verification process, manage availability, receive and accept ride offers, navigate to the pickup location, and perform the trip. Once the trip is completed, the platform calculates the final fare, coordinates payment and driver settlement, issues a receipt, and records the corresponding trip and earnings information. The platform also provides rating, feedback, support, and safety-case management capabilities.

The system therefore acts as the central coordination layer for a geographically distributed transportation marketplace. Its scope includes account and driver management, real-time matching and dispatch, trip lifecycle management, dynamic pricing, fare calculation and settlement orchestration, real-time tracking, ratings and reputation, support and safety workflows, notification orchestration, marketplace operations, and fraud/risk logic. External services such as payment processing, identity verification, mapping data, message delivery, user devices, and network connectivity remain outside the system boundary. This separation allows the platform to focus on the capabilities it directly controls while integrating with specialized external providers.

The platform is intended to operate at global scale. The workload model establishes a baseline of approximately 250,000 concurrent users under average conditions and more than 500,000 concurrent users during peak periods. The system must sustain approximately 45 million requests per hour under normal conditions and up to 120 million requests per hour during high-demand periods, with peak workloads lasting approximately three to four hours. These requirements are complemented by a target of 99.99

1.2 Key Distributed Systems Challenges

Building this platform from scratch is primarily a distributed-systems challenge because the system must coordinate a continuously changing, geographically distributed market-

place while maintaining different levels of correctness for different types of data.

First, driver availability and rider demand represent perishable, real-time resources. A driver who is idle in one location cannot be reused later for the same opportunity, and a rider's request must be matched while the driver and rider are still available. Consequently, matching and dispatch cannot rely on slow batch processing; they require low-latency, continuously updated state.

Second, the system must perform local geographic decisions at global scale. A ride request normally needs to consider drivers within a relatively small geographic area, but thousands of such decisions may occur simultaneously across cities and regions. The workload therefore requires efficient spatial indexing, rapid driver discovery, and geographically distributed processing. The workload model also assumes frequent driver telemetry updates, with active drivers periodically sending their current geographic position, creating a write-intensive workload.

Third, the platform must tolerate highly variable and bursty demand. Normal commuting periods can produce predictable increases in traffic, while events or other localized conditions can generate sudden surges concentrated in specific geographic areas. The system must therefore scale its processing capacity without significantly degrading matching latency, tracking, or other interactive operations.

Fourth, different components of the platform have different consistency requirements. Driver location and availability information prioritize freshness, low latency, and availability, and some temporary inconsistency may be tolerable. In contrast, trip state, payment processing, driver earnings, and settlement require stronger correctness guarantees. A duplicate payment or inconsistent settlement cannot be treated in the same way as a slightly stale location update. The architecture must therefore support multiple consistency and reliability requirements within the same overall system.

Fifth, the platform must remain operational despite failures in individual components or geographic regions. The target of 99.99

Finally, trust, safety, and security are fundamental system requirements rather than secondary features. The platform connects strangers and processes financial transactions, requiring driver verification, controlled access to trip information, ratings and reputation mechanisms, fraud detection, and support workflows for incidents and disputes. These capabilities must operate together with the real-time marketplace rather than being added independently after the core system is designed.

Overall, the system must solve the problem of coordinating a large, geographically distributed, real-time marketplace while simultaneously meeting strict requirements for latency, availability, scalability, correctness, and safety. The functional requirements, non-functional requirements, and workload model presented in the remainder of this document translate these challenges into measurable system requirements. These requirements provide the foundation for the subsequent architectural design and technology decisions

needed to build the platform from scratch.

2 System Description

This section sets out what the platform has to do and the conditions it has to work under, before any architectural or technological choice is made. As the delivery requires, it stays technology-agnostic: no cloud provider, database, or implementation choice is assumed. We begin with the business problem that motivates the system and why it becomes hard at scale, then move to the actors that interact with it, the main use cases those actors drive, and finally the boundary that separates the system from everything around it.

2.1 Business Problem

The pain point

City transportation has long carried a coordination failure between two groups that never see each other in time: people who need a ride at a given moment, and drivers with idle capacity at that same moment. Under the older model of street hailing and phone-dispatched taxis, neither side has reliable real-time information about the other. A rider cannot tell whether a car is nearby, how long it will take, or what the trip will cost until it is over; a driver has no clear picture of where unmet demand sits and ends up spending a good part of the day driving empty. The visible symptoms are long and unpredictable waits, poor vehicle use, opaque fares, cash-only friction, and little accountability on either safety or service.

Uber's answer is to run a two-sided marketplace that connects ride demand with independent driver supply in real time, without owning the cars. The platform closes the information gap between the two sides: it finds nearby drivers, shows an up-front price and an estimated arrival time, coordinates the trip from request to drop-off, and settles payment on its own. An informal, badly coordinated activity becomes a measurable on-demand service.

Who benefits

The value is different for each side of the market and for the environment around it. **Riders** get on-demand availability, predictable ETAs, transparent and cashless pricing (or cash, still common in much of Latin America), trip tracking, and safety mechanisms such as identity checks and a shareable trip status. **Drivers** get flexible earning opportunities and a demand signal that tells them where and when riders are requesting, which raises how much of their time and vehicle actually earns. **Cities and the wider economy**

gain a mobility option that complements public transit and can reduce the need to own a car for occasional trips.

The scale of this coordination is large. Uber reported 11.273 billion trips and \$162.8 billion in gross bookings for fiscal year 2024, and in the fourth quarter of 2024 it served 171 million monthly active platform consumers, an average of roughly 33 million trips a day [1]. The platform runs in around 70 countries and more than 10,000 cities [2]. Each of those trips is a separate, geographically local coordination event that has to resolve in seconds.

Why it is hard at scale

What makes Uber a genuine distributed-systems problem is not the raw volume on its own, but the set of constraints that have to hold at the same time for every one of those events:

1. **Perishable, real-time inventory.** The unit of supply is a driver-minute, and it cannot be stored: a minute a driver spends idle is gone. The market therefore has to clear continuously and in real time, not in batches, so that supply and demand meet while both still exist.
2. **Local decisions on a global system.** Matching is inherently local, since it concerns drivers within a few kilometers of a rider, yet these local decisions happen at once across thousands of cities. The system has to reason at fine geographic granularity and at very large aggregate volume. Uber’s H3 hexagonal index was built to bucket marketplace events by area so that supply, demand, and matching can be analyzed per zone in every city it serves [2].
3. **Latency as a product requirement.** A rider expects a nearby driver and an ETA within seconds of requesting, and driver locations stream in constantly and must stay fresh. How fast matching responds is part of the product, not an implementation detail.
4. **Dynamic market clearing.** When demand in an area outruns supply, price has to rise to rebalance the two sides, attracting more drivers and tempering demand, and it has to do so per area and in near real time. Uber derives this surge by measuring supply and demand within hexagonal cells in each city [2].
5. **Bursty, uneven load.** Demand follows time of day, weather, and events, so peaks (rush hours, concerts, New Year’s Eve) land far above the daily average and in particular places. The core matching experience has to hold up through those spikes.
6. **Different correctness needs in one request.** Parts of the platform sit at different points of the consistency–availability trade-off. Payments, driver payouts, and trip

state need strong correctness and a clear audit trail; money cannot be lost or charged twice. Location updates and driver discovery, on the other hand, favor low latency and high availability and can live with weaker, eventual consistency. A single request often touches both at once.

7. **Trust and safety as requirements, not features.** The platform puts strangers together and moves real money at scale, so identity verification, ratings, fraud prevention, and trip safety are load-bearing requirements that shape the whole design.
8. **Geographic and regulatory spread.** Users sit across many countries and rule sets, which brings both latency constraints (serving users near where they are) and functional variation (pricing rules, driver requirements, and payment methods differ by market).

Put together, these constraints frame the core engineering problem: keep clearing a perishable, two-sided, local marketplace, in real time and at global scale, while holding strong correctness for money and safety and high availability for matching. The requirements in the rest of this document, and the architecture decisions in the deliveries that follow, all trace back to this.

2.2 Actors

An actor is any entity, human or external system, that sits outside the system’s boundary and interacts with it to reach a goal. Two things follow from that definition. First, parties that are affected by the platform but never interact with it directly (cities, regulators, and the wider economy from Section 2.1) are indirect stakeholders, not actors, and do not appear here. Second, components that live inside the system (the matching, pricing, or fraud logic, for example) are not actors either; only entities the system does not own qualify.

We classify actors on two axes. By nature they are either human or system (external) actors. By role in an interaction, a *primary* actor starts a use case for its own goal, while a *secondary* actor is called on by the system to help finish one. For each actor we give its role, its typical interaction frequency, and what it expects of the system. Since this is a requirements-level view of the global platform, frequency is described in qualitative terms; the numbers behind it are left to the non-functional requirements and the workload model.

Human actors

- **Rider (Passenger) — primary.** The demand side. The rider requests a trip, is matched with a driver, travels, pays, and rates the trip. The interaction is occasional but active and latency-sensitive over the request–trip–payment cycle. A rider expects a

nearby driver within seconds, an accurate ETA and up-front price, a safe and trackable ride, easy payment (card or cash), and a real answer when something goes wrong.

- **Driver — primary.** The supply side. The driver goes online to offer availability, receives and accepts offers, drives to and carries the rider, and gets paid. While online the interaction is continuous: location streams constantly and offers arrive in real time. A driver expects a steady flow of nearby requests, transparent and timely earnings, workable navigation, a sense of where demand is, and support when needed.
- **Customer Support / Safety Agent — secondary.** A person who handles incidents, disputes, account problems, and safety escalations, including live help during a trip. The interaction is event-driven, triggered only when a rider or driver raises something, but it has to be available around the clock. The agent needs access to trip history and live trip state, plus the tools to act (refunds, account changes, escalation).
- **Operations / Market Administrator — secondary.** An internal operator responsible for a city or region: reviewing driver onboarding, setting pricing and service-area policy, and watching marketplace health. This is administrative and periodic work, not per-trip. The administrator needs dashboards and controls, the ability to set policy per market, and visibility into supply, demand, and service quality.

System (external) actors

- **Payment Service Provider — secondary.** Authorizes and captures rider charges and pays drivers out through the supported payment methods. It is called once per trip (authorization at the start, capture at completion) plus on the payout schedule. From it the system needs secure, reliable, idempotent transactions and a firm settlement result.
- **Mapping & Geospatial Data Provider — secondary.** Supplies the base map data and geocoding used to place riders and drivers, estimate ETAs, and drive navigation. Much of the routing and geospatial logic is built in-house and sits inside the boundary; the actor here is only the external base-map and geodata source. It is used continuously and at high volume, and the system needs accurate, low-latency responses from it.
- **Identity & Background Verification Provider — secondary.** Verifies driver identity and runs background screening at onboarding, with periodic re-screening and real-time identity checks thereafter [3]. It is used mostly at onboarding and on the re-check schedule, and the system needs authoritative, auditable results from it.
- **Notification / Communications Provider — secondary.** Delivers one-time passwords, push notifications, SMS, and email (trip updates, receipts). It is used often and is event-driven, with several messages per trip, and what the system needs from

it is timely, dependable delivery.

Actor catalog summary

Table 1 gathers the catalog for quick reference and for traceability in later deliveries.

Actor	Nature	Frequency	Key expectations
Rider (Passenger)	Human	Occasional, active	Fast matching, accurate ETA and price, safe trackable ride, easy payment, support
Driver	Human	Continuous while online	Steady nearby requests, transparent fair earnings, navigation, demand signal, safety
Support / Safety Agent	Human	Event-driven (24/7)	Access to trip data and tools to resolve issues and escalate safety events
Operations / Market Admin	Human	Periodic, administrative	Dashboards, per-market policy controls, marketplace visibility
Payment Service Provider	System	Per trip	Secure, reliable, idempotent transactions and settlement confirmation
Mapping & Geospatial Data	System	Continuous, high volume	Accurate, low-latency map, geocoding, and ETA data
Identity & Background Verif.	System	Onboarding + periodic	Authoritative, auditable identity and screening results
Notification Comms	System	Event-driven, frequent	Timely, reliable delivery of messages and one-time passwords

Table 1: Actor catalog for the Uber platform (global scope).

2.3 Main Use Cases

This subsection specifies the platform’s primary use cases, the ones that carry its core value. Supporting, exceptional, and purely administrative interactions are left out to keep the model focused. Each use case follows the same template: the actors involved (from the catalog in Section 2.2), preconditions, the main (success) flow, the relevant alternative flows, and postconditions. Each carries an identifier (UC-*nn*) so that functional requirements in later sections can trace back to it.

UC-01 — Request and match a ride

Primary actor

Rider (Passenger).

Secondary actors

Driver (supply side); Mapping & Geospatial Data Provider.

Preconditions

The rider is authenticated with a valid account and a registered payment method; the rider's location is available; the request channel is operational.

- Main flow**
1. The rider specifies pickup and destination.
 2. The system estimates the route, ETA, and up-front price for the available service tiers.
 3. The rider reviews the offer and confirms the request.
 4. The system finds nearby available drivers and dispatches the request according to its matching policy.
 5. A driver accepts the request.
 6. The system confirms the match to the rider, showing driver and vehicle details and the arrival ETA.

- Alternative flows**
- **A1 — No supply available.** No eligible driver is found nearby; the system tells the rider and may offer to keep searching or to notify when supply appears.
 - **A2 — Demand exceeds supply.** Dynamic pricing applies; the adjusted price is shown and the rider has to reconfirm before dispatch continues.
 - **A3 — Offer not accepted.** The chosen driver declines or does not respond in time; the system re-dispatches to the next candidate, or times out and informs the rider.
 - **A4 — Rider cancels.** The rider cancels before pickup; the system voids the pending trip and applies the cancellation policy if it applies.

Postconditions

On success, a trip is created in the “en route to pickup” state with a driver assigned, and both parties are notified. On failure, no trip is created and the rider is told why.

UC-02 — Accept and perform a trip

Primary actor

Driver.

Secondary actors

Rider; Mapping & Geospatial Data Provider.

Preconditions

The driver is onboarded, verified, and online; the driver's location is being shared; a dispatch offer has been generated for this driver by the matching step of UC-01.

- Main flow**
1. The system offers a nearby trip request to the driver.
 2. The driver accepts the offer.
 3. The system gives navigation to the pickup point, with rider status and contact.
 4. The driver reaches the pickup point and confirms pickup; the system starts the trip.
 5. The system gives navigation to the destination and tracks the trip in real time.
 6. The driver reaches the destination and ends the trip.

- Alternative flows**
- **A1 — Offer declined or ignored.** The offer expires and returns to UC-01 for re-dispatch to another driver.
 - **A2 — Rider no-show.** After a set wait, the driver cancels; the no-show or cancellation policy applies.
 - **A3 — Mid-trip change.** The rider asks for a stop or a new destination; the system updates route and fare accordingly.
 - **A4 — Trip interrupted.** The trip ends early after an incident; the fare is adjusted and support may step in.

Postconditions

On success, the trip reaches the “completed” state awaiting settlement and hands off to UC-03. On decline or cancellation, the driver returns to the available pool.

UC-03 — Complete trip and process payment

Primary actor

Rider (the billed party); *triggered by* trip completion in UC-02.

Secondary actors

Payment Service Provider; Driver (payout beneficiary).

Preconditions

A trip has been completed; the rider has a registered payment method or an agreed cash arrangement; the fare parameters are known.

- Main flow**
1. On completion, the system computes the final fare (base, distance and time, any dynamic pricing, tolls and fees, and promotions).
 2. The system asks the Payment Service Provider to authorize and capture the charge against the rider's payment method.
 3. The provider processes the charge and returns the result.
 4. The system issues a receipt to the rider and records the driver's earnings for the trip.
 5. The system schedules the driver payout per the payout policy.

- Alternative flows**
- **A1 — Payment declined.** The system records an outstanding balance, notifies the rider, and retries or asks for another method.
 - **A2 — Fare dispute.** Through UC-05, the fare is reviewed and, if warranted, corrected or refunded.
 - **A3 — Cash settlement (common in Latin American markets).** The fare is paid in cash directly; the system records the cash trip and reconciles balances instead of charging a card.

Postconditions

On success, the trip is settled, the rider has a receipt, and the driver's earnings are recorded and scheduled for payout. On failure, the trip is flagged with an outstanding balance for follow-up.

UC-04 — Onboard and verify a driver

Primary actor

Driver (applicant).

Secondary actors

Identity & Background Verification Provider; Operations / Market Administrator.

Preconditions

The applicant has created an account; the market's onboarding requirements are configured.

- Main flow**
1. The applicant submits the required identity, license, and vehicle information and documents.
 2. The system forwards identity verification and background screening to the verification provider.
 3. The provider returns the identity and screening results.

4. The system, and where policy requires it an Operations Administrator, reviews the results against market requirements.
5. On approval, the system activates the driver account, letting the driver go online.

Alternative flows • **A1 — Disqualifying result.** The application is rejected or held; the applicant is notified with the reason where permitted.

- **A2 — Incomplete documents.** The system asks for corrections and resubmission.

- **A3 — Periodic re-screening.** An active driver is re-verified on a schedule; a failure can suspend the account.

Postconditions

On approval, a verified, active driver exists and can accept trips (enabling UC-02). On rejection, no active driver account is created.

UC-05 — Rate and report a trip

Primary actor

Rider or Driver (either may rate the other).

Secondary actors

Support / Safety Agent.

Preconditions

A trip has been completed (or, for a safety report, is in progress or just ended); the participant is authenticated.

- Main flow**
1. After the trip, the system prompts the participant to rate the other party and, optionally, leave feedback.
 2. The participant submits a rating and any optional comment or tip.
 3. The system records the rating and updates the other party's aggregate score.
 4. If the participant files a report (safety, service, or lost item), the system routes it to a Support / Safety Agent.
 5. The agent reviews and resolves the report, taking the appropriate action per policy.

Alternative flows • **A1 — No rating submitted.** The system moves on; the rating may be asked for again later or skipped.

- **A2 — Low rating or serious report.** The trip is flagged for review and may lead to follow-up or account action.
- **A3 — Lost item or dispute.** The case is routed to support with the relevant trip data for resolution.

Postconditions

Ratings are recorded and aggregates updated; any report carries a tracked resolution state, and quality and safety signals become available for later policy.

2.4 System Boundaries

This subsection sets the limits of the system: what sits inside its boundary (the capabilities it owns and answers for), what is explicitly out of scope, and the external interfaces through which it meets its environment. The scope is the global Uber ride-sharing platform, in line with the worldwide figures in Section 2.1; where a Latin American market differs in a way that matters, it is noted. The test throughout is one of control: a capability the platform designs, runs, and answers for is inside the boundary; a capability it depends on but does not control is outside it.

Inside the boundary (in scope)

The system owns the following:

- Rider and driver account management, profiles, and session authentication.
- Real-time supply–demand **matching and dispatch** (candidate discovery, offer, and assignment).
- **Trip lifecycle management** across its states (requested, en route to pickup, in progress, completed, settled).
- **Dynamic pricing** (surge) computation per geographic area.
- **Fare calculation and settlement orchestration:** computing fares and coordinating charges and payouts (the payment execution itself is external).
- In-house geospatial and routing logic (ETA estimation, trip tracking) built on external base-map data.
- **Ratings, feedback, and reputation** management.
- Support-case management and safety-incident routing (the workflow, not third-party services).

- Notification orchestration: deciding what to send and when (message delivery itself is external).
- Marketplace configuration and operations tooling: per-market policy, service areas, and monitoring.
- Risk and fraud logic.

LATAM note. Settlement orchestration has to handle **cash-paid trips**, which are common in several Latin American markets; for those trips the platform records and reconciles the fare rather than relying on an external card charge.

Outside the boundary (out of scope)

The following are not built or owned by the system; the platform depends on or coexists with them:

- Payment processing, banking, and settlement, handled by an external payment service provider.
- The actual identity verification and background screening (an external verification provider).
- Base map data and geocoding sources (an external mapping/geodata provider).
- Message-delivery infrastructure for SMS, push, and email (external carriers).
- Rider and driver devices, their operating systems, and network connectivity.
- The physical transportation itself, the vehicles, and their maintenance.
- Regulatory, tax, and compliance rule-making (the platform complies with rules; it does not write them).

External interfaces

Every actor in the catalog (Table 1) meets the platform across the boundary, and each crossing is an integration point. The human actors (rider, driver, support/safety agent, operations administrator) start or service interactions with the platform. The external systems are called on by the platform: the payment provider for authorization, capture, and payouts; the mapping/geodata provider for base-map and geocoding data; the identity and background verification provider for onboarding and periodic checks; and the notification provider for message delivery. These crossings are drawn in Figure 1.

System boundary diagram

Figure 1 shows the system as a single black box: it shows what the platform interacts with, not how it is built inside. Opening the box into services is left, on purpose, to the software-architecture delivery.

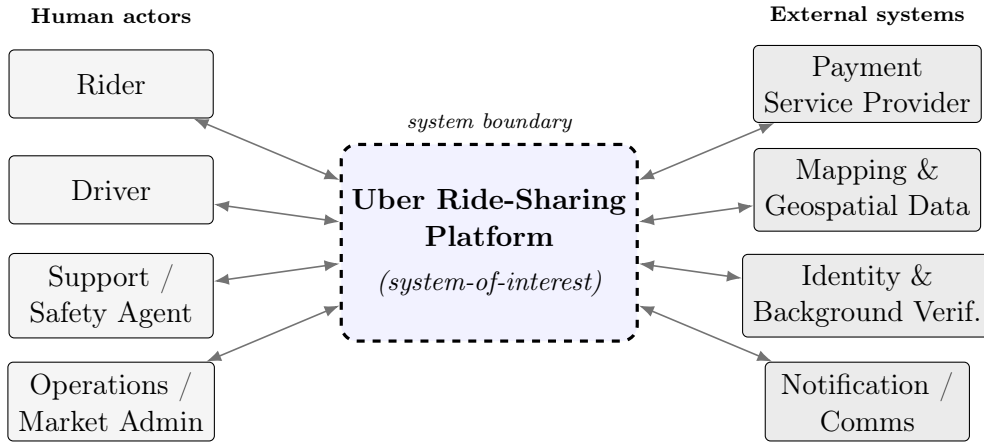


Figure 1: System boundary (context) diagram for the global Uber ride-sharing platform. The platform is shown as a black box; arrows are interactions crossing the boundary, not internal structure.

3 Functional Requirements

This section defines the functional requirements of the Uber ride-sharing platform. The requirements describe the main capabilities that the system shall provide to its actors and are organized by functional domain. Each requirement has a unique identifier and is traceable to one or more of the main use cases defined in the System Description.

3.1 Account and Driver Management

ID	Requirement	Priority	Traceability
FR-01	The system shall onboard and verify driver applicants before allowing them to accept ride requests.	High	UC-04 – Onboard and Verify a Driver

ID	Requirement	Priority	Traceability
FR-02	The system shall manage the availability state of verified drivers according to their online, offline, available, assigned, and active-trip states.	High	UC-02 – Accept and Perform a Trip

Table 2: Functional requirements for account and driver management

3.1.1 FR-01 – Driver Onboarding and Verification

Field	Specification
Requirement	The system shall onboard and verify driver applicants before allowing them to accept ride requests.
Inputs	Driver identity information, driver’s license information, vehicle information, required documents, and verification results.
Source	Driver applicant; Identity and Background Verification Provider; Operations / Market Administrator.
Output	Verified or rejected driver status, onboarding decision, and corresponding notification.
Destination	Driver; Operations / Market Administrator; driver availability management.
Constraints	A driver shall not be allowed to accept trips until the required verification and applicable market requirements have been successfully completed.
Process	The system shall receive the driver’s information and documents, submit the required information for identity and background verification, evaluate the returned results against the applicable requirements, and activate the driver account when the verification process is successful.

Table 3: Specification details for driver onboarding and verification requirement (FR-01)

3.1.2 FR-02 – Driver Availability Management

Field	Specification
Requirement	The system shall manage the availability state of verified drivers according to their online, offline, available, assigned, and active-trip states.
Inputs	Driver availability changes, driver location updates, trip assignments, trip acceptance or rejection, and trip completion or cancellation events.
Source	Driver; trip management; matching and dispatch functions.
Output	Current driver availability state and eligibility for ride matching.
Destination	Matching and dispatch functions; driver application; operations functions.
Constraints	Only verified drivers in an eligible state shall be considered for ride assignment. A driver assigned to an active trip shall not be simultaneously assigned to another trip.
Process	The system shall update the driver’s state according to availability events and trip lifecycle events, maintain the driver’s current location, and make eligible drivers available to the matching process.
Traceability	UC-02 – Accept and Perform a Trip.

Table 4: Specification details for driver availability management requirement (FR-02)

3.2 Ride Request and Matching

ID	Requirement	Priority	Traceability
FR-03	The system shall create a ride request using the rider's pickup location, destination, selected service type, and applicable request information.	High	UC-01 – Request and Match a Ride
FR-04	The system shall identify and assign an eligible nearby driver to a confirmed ride request according to the matching policy.	Critical	UC-01 – Request and Match a Ride
FR-05	The system shall calculate and present an estimated fare and arrival time for a ride request before the rider confirms the request.	High	UC-01 – Request and Match a Ride

Table 5: Functional requirements for ride requests and matching

3.2.1 FR-03 – Ride Request Creation

Field	Specification
Requirement	The system shall create a ride request using the rider's pickup location, destination, selected service type, and applicable request information.
Inputs	Rider identity, pickup location, destination, selected service type, and request confirmation.
Source	Rider.
Output	A validated ride request containing the required trip information and an initial request state.
Destination	Matching and dispatch functions; rider.

Field	Specification
Constraints	The rider shall be authenticated and the required trip information shall be available before the request is created.
Process	The system shall receive the rider's pickup and destination information, validate the request data, calculate the applicable trip information, and create the ride request after rider confirmation.
Traceability	UC-01 – Request and Match a Ride.

Table 6: Specification details for ride request creation requirement (FR-03)

3.2.2 FR-04 – Driver Matching and Assignment

Field	Specification
Requirement	The system shall identify and assign an eligible nearby driver to a confirmed ride request according to the matching policy.
Inputs	Confirmed ride request, driver availability state, driver location, service type, and matching criteria.
Source	Ride request; driver availability management; Mapping and Geospatial Data Provider.
Output	Driver assignment, updated trip state, and driver and rider notifications.
Destination	Assigned driver; rider; trip management.
Constraints	Only eligible and available drivers shall be considered. If a driver declines or does not respond within the applicable period, the system shall be able to continue the dispatch process with another eligible candidate.
Process	The system shall identify eligible nearby drivers, evaluate candidates according to the matching policy, dispatch the request to a selected candidate, process the driver's response, and confirm the assignment when the request is accepted.

Traceability	UC-01 – Request and Match a Ride.
--------------	-----------------------------------

Table 7: Specification details for driver matching and assignment requirement (FR-04)

3.2.3 FR-05 – Fare and ETA Estimation

Field	Specification
Requirement	The system shall calculate and present an estimated fare and arrival time for a ride request before the rider confirms the request.
Inputs	Pickup location, destination, service type, route information, estimated travel time, applicable pricing parameters, and dynamic pricing conditions.
Source	Rider; Mapping and Geospatial Data Provider; marketplace pricing data.
Output	Estimated route, estimated arrival time, and estimated ride price.
Destination	Rider; ride request process.
Constraints	The estimate shall be generated using the current trip information and applicable pricing and service-area rules.
Process	The system shall obtain or calculate route and travel-time information, apply the applicable fare parameters and dynamic pricing conditions, and present the resulting estimate to the rider before request confirmation.
Traceability	UC-01 – Request and Match a Ride.

Table 8: Specification details for fare and ETA estimation requirement (FR-05)

3.3 Trip Management

ID	Requirement	Priority	Traceability
FR-06	The system shall manage the lifecycle of a trip from confirmed assignment through pickup, active travel, completion, cancellation, or interruption.	Critical	UC-01 – Request and Match a Ride; UC-02 – Accept and Perform a Trip
FR-07	The system shall provide real-time trip tracking using current driver location and trip state information while a trip is active.	Critical	UC-02 – Accept and Perform a Trip

Table 9: Functional requirements for trip management

3.3.1 FR-06 – Trip Lifecycle Management

Field	Specification
Requirement	The system shall manage the lifecycle of a trip from confirmed assignment through pickup, active travel, completion, cancellation, or interruption.
Inputs	Ride assignment, driver acceptance, pickup confirmation, trip status events, cancellation events, destination changes, and trip completion information.
Source	Rider; Driver; matching and dispatch functions; Mapping and Geospatial Data Provider.
Output	Updated trip state and trip information available to the involved actors and subsequent processes.
Destination	Rider; Driver; payment and settlement process; support and safety functions.
Constraints	Trip state transitions shall follow the valid lifecycle sequence. Cancellation, interruption, and destination-change events shall be processed according to the applicable trip policies.

Field	Specification
Process	The system shall create the trip after a successful match, update its state as the driver approaches pickup and the trip progresses, record completion or exceptional events, and transition the trip to the appropriate final state.
Traceability	UC-01 – Request and Match a Ride; UC-02 – Accept and Perform a Trip.

Table 10: Specification details for trip lifecycle management requirement (FR-06)

3.3.2 FR-07 – Real-Time Trip Tracking

Field	Specification
Requirement	The system shall provide real-time trip tracking using current driver location and trip state information while a trip is active.
Inputs	Driver geographic coordinates, trip state, route information, and destination information.
Source	Driver; Mapping and Geospatial Data Provider.
Output	Updated driver location, route progress, estimated arrival information, and current trip state.
Destination	Rider; Driver; support and safety functions.
Constraints	Location information shall be associated with the correct active trip and shall only be used for authorized trip participants and operational functions.
Process	The system shall receive driver location updates during an active trip, associate the updates with the corresponding trip, update route progress and estimated arrival information, and make the resulting state available to authorized participants.
Traceability	UC-02 – Accept and Perform a Trip.

Table 11: Specification details for real-time trip tracking requirement (FR-07)

3.4 Payment and Settlement

ID	Requirement	Priority	Traceability
FR-08	The system shall calculate the final trip fare and coordinate payment processing and driver settlement after a trip is completed.	Critical	UC-03 – Complete Trip and Process Payment

Table 12: Functional requirements for payment and settlement

3.4.1 FR-08 – Fare Calculation and Payment Processing

Field	Specification
Requirement	The system shall calculate the final trip fare and coordinate payment processing and driver settlement after a trip is completed.
Inputs	Completed trip information, distance, travel time, pricing parameters, dynamic pricing conditions, tolls, fees, promotions, rider payment method, and payment provider result.
Source	Trip management; Rider; Payment Service Provider; pricing and settlement data.
Output	Final fare, payment status, receipt, recorded driver earnings, and applicable payout status.
Destination	Rider; Driver; Payment Service Provider; settlement records.
Constraints	Payment operations shall be associated with the correct trip and shall prevent duplicate settlement of the same transaction. Cash-paid trips shall be recorded and reconciled according to the applicable process.

Field	Specification
Process	The system shall calculate the final fare when the trip is completed, request payment authorization and capture when applicable, record the payment result, issue a receipt, record the driver's earnings, and schedule or reconcile the corresponding settlement.
Traceability	UC-03 – Complete Trip and Process Payment.

Table 13: Specification details for fare calculation and payment processing requirement (FR-08)

3.5 Feedback and Safety

ID	Requirement	Priority	Traceability
FR-09	The system shall record ratings and feedback associated with completed trips from eligible trip participants.	Medium	UC-05 – Rate and Report a Trip
FR-10	The system shall create and manage support or safety cases associated with reported trip incidents or disputes.	High	UC-05 – Rate and Report a Trip

Table 14: Functional requirements for feedback and safety

3.5.1 FR-09 – Rating and Feedback Management

Field	Specification
Requirement	The system shall record ratings and feedback associated with completed trips from eligible trip participants.
Inputs	Trip identifier, actor identity, rating, optional feedback, and trip outcome information.

Field	Specification
Source	Rider; Driver.
Output	Recorded rating and feedback associated with the corresponding trip and participant.
Destination	Rating and reputation records; authorized operations and support functions.
Constraints	Only eligible participants associated with the trip shall be allowed to submit feedback for that trip.
Process	The system shall validate the participant's eligibility, receive the rating and optional feedback, associate the information with the corresponding completed trip, and store the resulting feedback record.
Traceability	UC-05 – Rate and Report a Trip.

Table 15: Specification details for rating and feedback management requirement (FR-09)

3.5.2 FR-10 – Safety and Support Case Management

Field	Specification
Requirement	The system shall create and manage support or safety cases associated with reported trip incidents or disputes.
Inputs	Trip identifier, incident or dispute description, actor identity, supporting information, and case updates.
Source	Rider; Driver; Support / Safety Agent.
Output	Support or safety case, case status, assigned handling information, and resolution record.
Destination	Support / Safety Agent; Rider; Driver; authorized operations functions.
Constraints	Case information shall be associated with the relevant trip or account and shall only be accessible to authorized actors.

Field	Specification
Process	The system shall receive the reported incident or dispute, create a case, associate the case with the relevant trip and actors, provide the required information to authorized support or safety personnel, and record case updates and resolution.
Traceability	UC-05 – Rate and Report a Trip.

Table 16: Specification details for safety and support case management requirement (FR-10)

3.6 Functional Requirements Summary

Table 17 summarizes the ten functional requirements and their traceability to the main use cases.

ID	Functional Area	Use Case Traceability
FR-01	Driver Onboarding and Verification	UC-04
FR-02	Driver Availability Management	UC-02
FR-03	Ride Request Creation	UC-01
FR-04	Driver Matching and Assignment	UC-01
FR-05	Fare and ETA Estimation	UC-01
FR-06	Trip Lifecycle Management	UC-01, UC-02
FR-07	Real-Time Trip Tracking	UC-02
FR-08	Fare Calculation and Payment Processing	UC-03
FR-09	Rating and Feedback Management	UC-05
FR-10	Safety and Support Case Management	UC-05

Table 17: Functional requirements summary

4 Non-Functional Requirements

This section defines the measurable non-functional requirements of the Uber ride-sharing platform. The requirements establish the expected operational characteristics of the system in terms of workload capacity, performance, availability, data management, and geographic scope.

The quantitative targets are consistent with the workload parameters defined in the Workload Model in Section 5. The workload model establishes an average request rate of 45,000,000 requests per hour, a peak request rate of 120,000,000 requests per hour, peak periods lasting approximately 3 to 4 hours, 250,000 concurrent users on average, and more than 500,000 concurrent users during peak conditions. It also defines a 99.99% availability target, a p95 response-time target below 500 ms, an estimated initial data volume of 20 TB, annual data growth of 15 TB, and global geographic scope.

4.1 Performance and Capacity

4.1.1 NFR-01 – Expected Users

Field	Specification
Category	Expected Users / Capacity
Requirement	The system shall support approximately 250,000 concurrent users under average operating conditions and at least 500,000 concurrent users during peak operating conditions.
Metric	Number of concurrent active users supported by the system.
Target	250,000 concurrent users on average and 500,000+ concurrent users at peak.
Verification	Load testing shall demonstrate that the system can sustain the defined average and peak concurrent-user levels while maintaining the response-time and availability targets defined in this section.
Source	Workload Model, Section 5.1.

Table 18: Non-functional requirement specification for expected concurrent users (NFR-01)

4.1.2 NFR-02 – Average Request Rate

Field	Specification
Category	Average Requests per Hour
Requirement	The system shall sustain an average workload of 45,000,000 requests per hour during normal operating conditions.

Field	Specification
Metric	Requests processed per hour under average operating conditions.
Target	45,000,000 requests/hour.
Verification	A sustained-load test shall generate an equivalent request rate and verify that the system continues to meet the response-time and availability targets.
Source	Workload Model, Section 5.1.

Table 19: Non-functional requirement specification for average request rate (NFR-02)

4.1.3 NFR-03 – Peak Request Rate

Field	Specification
Category	Peak Requests per Hour
Requirement	The system shall sustain a peak workload of 120,000,000 requests per hour during high-demand operating periods.
Metric	Maximum expected request-processing rate during peak conditions.
Target	120,000,000 requests/hour.
Verification	A peak-load test shall generate the target request rate and verify successful request processing without violating the response-time and availability requirements.
Source	Workload Model, Section 5.1.

Table 20: Non-functional requirement specification for peak request rate (NFR-03)

4.1.4 NFR-04 – Peak Load Duration

Field	Specification
Category	Peak Duration

Field	Specification
Requirement	The system shall sustain the defined peak workload of 120,000,000 requests per hour for a continuous period of 3 to 4 hours.
Metric	Continuous duration for which the peak request rate can be sustained.
Target	3–4 consecutive hours.
Verification	A stress or endurance test shall maintain the peak workload for the specified duration and verify that the system remains operational and continues to meet the defined response-time and availability targets.
Source	Workload Model, Section 5.1.

Table 21: Non-functional requirement specification for peak load duration (NFR-04)

4.1.5 NFR-05 – Response Time

Field	Specification
Category	Response-Time Target
Requirement	The system shall process latency-sensitive interactive requests with a response time below 500 milliseconds at the 95th percentile under the defined operating workload.
Metric	95th-percentile response time (p95).
Target	$p95 < 500 \text{ ms}$.
Verification	Performance testing shall measure end-to-end response times under average and peak workloads and verify that the p95 response time remains below 500 ms for the applicable interactive operations.
Source	Workload Model, Section 5.1.

Table 22: Non-functional requirement specification for target response time (NFR-05)

4.2 Availability and Reliability

4.2.1 NFR-06 – System Availability

Field	Specification
Category	Availability
Requirement	The system shall provide a minimum availability of 99.99% over a yearly operating period.
Metric	Percentage of time during which the system is available to process required operations.
Target	99.99% annual availability, corresponding to a maximum of approximately 52.6 minutes of downtime per year.
Verification	Availability shall be measured using continuous service monitoring over the defined operating period, including planned and unplanned service interruptions according to the applicable availability measurement policy.
Source	Workload Model, Section 5.1.

Table 23: Non-functional requirement specification for system availability (NFR-06)

4.3 Data Management

4.3.1 NFR-07 – Initial Data Volume

Field	Specification
Category	Data Volume
Requirement	The system shall support an estimated initial data volume of 20 TB covering operational data required by the platform at launch.
Metric	Total stored data volume at system launch.
Target	20 TB.
Verification	The storage capacity and data-management design shall be evaluated against a baseline dataset representing the estimated launch volume of 20 TB.
Source	Workload Model, Section 5.1.

Field	Specification
-------	---------------

Table 24: Non-functional requirement specification for initial data volume (NFR-07)

4.3.2 NFR-08 – Annual Data Growth

Field	Specification
Category	Data Growth
Requirement	The system shall accommodate an estimated annual data growth of 15 TB without loss of required system functionality or degradation beyond the defined performance targets.
Metric	Additional stored data volume generated per year.
Target	15 TB/year.
Verification	Capacity and storage-management testing shall evaluate the system with the projected annual data increase and verify that the defined functional and performance requirements remain achievable.
Source	Workload Model, Section 5.1.

Table 25: Non-functional requirement specification for annual data growth (NFR-08)

4.4 Geographic Scope

4.4.1 NFR-09 – Global Geographic Coverage

Field	Specification
Category	Geographic Scope
Requirement	The system shall support ride-sharing operations across a global geographic scope while maintaining the required service availability and performance characteristics for users in different geographic regions.

Field	Specification
Metric	Geographic coverage of supported operations and ability to serve users across multiple regions.
Target	Global geographic scope.
Verification	The system shall be evaluated using representative geographically distributed workloads and shall demonstrate that regional operation does not prevent compliance with the defined availability, capacity, and response-time requirements.
Source	Workload Model, Section 5.1.

Table 26: Non-functional requirement specification for global geographic coverage (NFR-09)

4.5 Non-Functional Requirements Summary

Table 27 summarizes the measurable non-functional requirements and their corresponding targets.

ID	Category	Requirement	Target
NFR-01	Expected Users	Concurrent users supported under average and peak conditions	250,000 average; 500,000+ peak
NFR-02	Average Requests/Hour	Requests sustained under normal operating conditions	45,000,000 requests/hour
NFR-03	Peak Requests/Hour	Requests sustained during peak conditions	120,000,000 requests/hour
NFR-04	Peak Duration	Continuous duration of peak workload	3–4 hours
NFR-05	Response Time	95th-percentile response time	p95 < 500 ms
NFR-06	Availability	Annual system availability	99.99%

ID	Category	Requirement	Target
NFR-07	Data Volume	Estimated data volume at launch	20 TB
NFR-08	Data Growth	Estimated annual data growth	15 TB/year
NFR-09	Geographic Scope	Geographic coverage of the platform	Global

Table 27: Non-functional requirements summary

5 Workload Model

To accurately design the cloud-native architecture for the ride-sharing platform, we have synthesized the application’s workload characteristics based on project constraints and official Uber Engineering references. The system must support over 100,000 active rides simultaneously across multiple geographies and manage high-frequency database writes.

5.1 Workload Parameters

Table 28: Workload Parameters

Parameter	Value	Justification	Reference
Average re-requests/hour	45,000,000	Based on continuous GPS telemetry pings from active drivers every 4 seconds.	[4]
Peak requests/hour	120,000,000	Occurs during high-demand periods triggering surge pricing.	[5]
Peak duration	3 to 4 hours	Standard morning (7-10 AM) and evening (5-8 PM) commute rushes.	[1]
Read/Write ratio	40% Read / 60% Write	Heavy write volume due to real-time geolocation updates from drivers.	Project Baseline
Availability target (SLA)	99.99%	Critical requirement allowing a maximum downtime of 52.6 minutes/year.	Project Baseline
Response time target	p95 < 500 ms	Sub-second response times required for microsecond matching algorithms.	Project Baseline
Estimated data volume	20 TB	Storage for user profiles, payment tokens, and trip histories at launch.	[6]
Annual data growth	15 TB/year	Driven by trip receipts, audit trails, and historical telemetry data.	[6]
Concurrent users (avg)	250,000	Includes active riders, idle drivers, and drivers en route.	[4]
Concurrent users (peak)	500,000+	Supports the baseline of 100,000+ simultaneous active rides.	[1]
Geographic scope	Global	Requires geographic redundancy and state coordination across availability zones.	Project Baseline

5.2 Workload Assumptions

The workload model is built upon the following technical and business assumptions:

- **Telemetry Ingestion:** Every active driver app sends a geographic coordinate update via a WebSocket or streaming pipeline every 3 to 4 seconds, creating an intensive write-heavy database profile [4].

- **Matching Algorithm Execution:** The dispatch system evaluates thousands of potential rider-driver pairs using spatial indexing (Uber’s H3) within microseconds [2].
- **State Management:** Distributed state across availability zones is required to handle network partitions without losing active trip data or interrupting payment coordination.

5.3 Workload Scenarios

To ensure resilience, the architecture must handle distinct operational profiles:

- **Normal Commute Scenario:** Characterized by predictable spikes between 7:00 AM - 9:00 AM and 5:00 PM - 7:00 PM. The system processes a steady stream of requests with standard dynamic pricing.
- **Extreme Surge Scenario:** A sudden, massive influx of concurrent searches and ride requests concentrated in specific urban zones. The system must instantly scale compute resources to handle surge pricing computation and real-time event streaming without degrading the sub-second response times [5].
- **Regional Outage Scenario:** If a primary availability zone experiences a failure, the geographic redundancy mechanisms must instantly route traffic to a secondary zone to preserve the 99.99% availability SLA.

References

- [1] Uber Technologies, Inc., “Uber Announces Results for Fourth Quarter and Full Year 2024,” *Uber Investor Relations*, Feb. 2025. [Online]. Available: <https://investor.uber.com/news-events/news/press-release-details/2025/Uber-Announces-Results-for-Fourth-Quarter-and-Full-Year-2024/default.aspx> (accessed Sep. 21, 2026).
- [2] Uber Engineering, “H3: Uber’s Hexagonal Hierarchical Spatial Index,” *Uber Engineering Blog*, 2018. [Online]. Available: <https://www.uber.com/us/en/blog/h3/> (accessed Sep. 21, 2026).
- [3] Uber Technologies, Inc., “How Driver Screenings Work — Safety,” *Uber Safety*. [Online]. Available: <https://www.uber.com/us/en/ride/safety/driver-screening/> (accessed Sep. 22, 2026).

- [4] Uber Engineering, “How Uber Scales Its Real-Time Infrastructure,” *Uber Engineering Blog*. [Online]. Available: <https://www.uber.com/en-US/blog/how-uber-scales-its-real-time-infrastructure/> (accessed Sep. 22, 2026).
- [5] Uber Engineering, “The Architecture of Uber’s API gateway,” *Uber Engineering Blog*. [Online]. Available: <https://www.uber.com/en-US/blog/architecture-api-gateway/> (accessed Sep. 22, 2026).
- [6] Uber Engineering, “Uber’s Big Data Platform: 100+ Petabytes with Minute Latency,” *Uber Engineering Blog*. [Online]. Available: <https://www.uber.com/en-US/blog/uber-big-data-platform/> (accessed Sep. 22, 2026).